

No Trading Strategy Can Win on Every Price Path

A Computable Countermarket for Every Trader

Karl Svozil
ITP, TU Wien, Vienna, Austria

ECONVN 2026/2027, HUB/AJEB,
Ho Chi Minh City, Vietnam
12-14 October 2026

<https://svozil.github.io/publications/2026-ECONVN-pres.pdf>
based on
<https://doi.org/10.48550/arXiv.2604.13334>

“Message”: Finite profits are possible, but not programmable

A trader can make money over a finite period

Accurate information, luck, or a continuing trend can produce profits. Exuberance, innovation, and improvements in production can support a rise.

The same kind of bet can also lose

Information may be false. Exuberance may turn into panic. Expected gains from innovation or cost cutting may fail to reach shareholders.

The claim that fails

“One trading rule that earns strict profit on every possible price history.”

The reason is constructive (algorithmically provable): use the proposed rule to build its *countermarket*, a price record on which it cannot win.

The trader chooses from the history so far

A trading rule

The rule observes past and current prices and chooses a position before the next price arrives. It cannot use a future price that it has not observed.

Three choices

Long: own shares and gain when their price rises.

Short: borrow and sell shares, intending to buy them back later; gain when their price falls.

Wait: hold no shares and earn zero trading gain.

The mathematical setting

The rule is an algorithm that returns an action on every finite history (*total and computable*), without random choices (*deterministic*). Positions are whole shares or fractions of integers (*rational* amounts).

Context: Cantor's diagonal idea, also used by Gödel, Turing et al

Proof by contradiction: Suppose there exist a *complete* list claimed to contain *all* infinite sequences of zeros and ones. Its beginning might look like this:

	bit 1	bit 2	bit 3	bit 4	...
row 1	0	1	1	0	...
row 2	1	1	0	1	...
row 3	0	1	0	0	...
row 4	1	0	1	1	...
⋮	⋮	⋮	⋮	⋮	⋮

Change the diagonal entries

Read the highlighted bits and change each 0 to 1, each 1 to 0. The new sequence begins 1, 0, 1, 0, ...

Why the list cannot be complete

The new sequence differs from row 1 at bit 1, from row 2 at bit 2, and from every row at its diagonal position. It is absent from the list.

Sources: Metamathematics; eg, Noson S. Yanofsky, *A Universal Approach to Self-Referential Paradoxes, Incompleteness and Fixed Points* (2003).

Discounted price: value compared with the bank

Definition

A *discounted price* is the asset's price divided by the current value of one unit of the comparison bank account. It tells us how many such bank units the asset is worth. The model treats this account as risk-free. All money amounts below are in a fixed currency; eg, Vietnamese dong (VND).

Example: one bank unit is a deposit of 100 VND

Suppose the account earns 10% over the year, whereas the share sells at 121 VND.

	Bank unit	One share	Discounted price
At the start	100	100	$100/100 = 1$
After one year	110	121	$121/110 = 1.10$

What the number means

Selling the share now buys 1.10 bank units instead of 1. The share gained 10% *relative to the bank account*. Had the share risen only to 110, its discounted price would still be 1. (This adjusts for bank interest, not inflation.)

From Cantor's idea to a countermarket

Cantor's construction

Start with a purportedly complete enumeration.

Use that list itself to construct something that it misses.

Changing the selected bit defeats the completeness claim.

The trading construction

Start with a proposed universally profitable rule.

Use that rule itself to construct a price path on which it fails.

Opposing each position defeats the universal-profit claim.

The common idea is diagonal opposition

The counterexample depends on the proposed universal object. Here we construct a market for one fixed trader; we do not need to list all trading programs.

A Computable Countermarket for Every Trader

The central result developed in the paper

For every deterministic trading program that returns a whole or fractional share position on every finite history, there is a fixed, positive, computable price path on which its total trading gain is never positive (price strictly above zero) at any finite date.

The stronger statement about active trading

Every nonzero position loses on the constructed path. After any period of active trading, its accumulated gain is strictly negative. A trader that always waits earns zero.

What we are measuring

Prices are discounted as on slide 5: gain is measured relative to the bank account. Trading is *self-financing*: new positions use the shares and bank balance already held, with no outside cash added. The simple proof omits trading fees, which would further reduce gains.

Sources: KS, *No Trading Strategy Can Win on Every Price Path*, section “A Computable Countermarket for Every Trader” (2026).

The countermarket: a constructive proof

Use the proposed trader's own program

Calculate its position from the history constructed so far. Then choose the next price move according to this table:

Opposition at every trading date

Trader's choice	Countermarket's move	Trading result
Hold or buy shares	Price falls	Loss
Sell borrowed shares	Price rises	Loss
Hold no position	Price stays the same	Zero

Why this proves the claim

Choose a fixed, small percentage rise or fall that keeps prices positive, and repeat. Adding losses and zeros can never produce a positive gain. For each fixed trader this gives one fixed computable price path, whereby the countermarket is constructed from this trader's program. Any finite part of its price history can be calculated before trading begins. It need not defeat other traders.

Worked example: we construct the adverse prices

The trader's rule is fixed first

The rule is: buy for one period, sell short for the next, then wait. **We construct a price history that makes this rule lose.**

Our construction, step by step

1. The trader buys one share for 100.
We choose 90 as the next price. Selling the share then loses 10.
2. The trader borrows a share and sells it for 90.
We choose 99 as the next price. Buying it back to return it loses 9.
3. The trader holds no shares.
We leave the price at 99. Its trading gain is zero.

Why this is a counterexample

The proposed rule claims a strictly positive gain on every price history. The history we have constructed instead produces a loss. Therefore, this rule cannot satisfy its claim of universal profitability.

Why the countermarket is a fixed computable path

Put the trader's code inside a price-generating program

1. Start with a positive price and the history built so far.
2. Run the trader's program on that history to obtain its position.
3. Choose a small price fall for a long position, a small rise for a short position, and no change for zero holdings.
4. Append that price to the history and repeat.

Each requested finite part can be calculated

The trader returns its position in finite time. Whole shares and fractions of integers can be compared with zero exactly. Each next price is therefore computable.

The market does not have to observe live orders

For a fixed trader, the generator is fixed. Any finite portion can be computed before trading begins, although computation may be slow.

Every trader has its own countermarket

The order of the claim matters

Proposed guarantee: “There is one trader that profits on every path.”

Countermarket result: “For each proposed trader, construct a path on which it fails to profit.”

Another trader may win on that same record

For one price rise, a trader holding one share gains and a trader shorting one share loses. A price fall reverses their fortunes. There need not be one path defeating all traders simultaneously.

The precise scope of the construction

The countermarket need not be typical, likely, or an economic equilibrium. Its role is to refute a claim about *every* positive computable price path. The argument uses direct diagonal opposition and needs no Halting-Problem reduction. The following results address separate questions.

Randomized trading: what does the trader randomize?

A private coin chooses the position

After observing the past, the trader may toss a private coin: heads means own one share, tails means sell one borrowed share. The coin chooses the *position*. The subsequent price movement determines whether that position gains or loses.

Why the earlier diagonal construction needs qualification

For a deterministic trader, its program and the price history determine its next position. With private randomness, we also need the outcome of its coin toss. The preceding countermarket construction therefore cannot calculate and oppose each hidden choice from the program and price history alone.

A separate argument using an independent market

We instead let the market choose its price movements independently of the trader's orders. Such a market is called *passive*: it does not react to those orders. The trader may still randomize, so both sides can randomize simultaneously.

We now calculate *expected gain*: the probability-weighted average of the possible gains and losses.

An independent fair market gives zero expected gain

The market's coin chooses the price movement

The trader first chooses its position, possibly at random. An independent fair coin then moves the discounted price from 100 to 90 or 110, each with probability one half. The market's coin is independent of the past and of the trader's private coin.

Average over the market's two equally likely moves

Position	Price 90	Price 110	Expected gain
Own one share	-10	+10	0
Short one share	+10	-10	0
Hold no position	0	0	0

The result applies to deterministic and randomized traders

Every chosen position has expected gain zero, however the trader selected it. Repeat with independent fair choices of a 10% rise or fall from each current price. Expected total gain over any fixed finite number of trades is then zero, provided gains and losses have finite averages.

Individual runs may win or lose. Randomizing the position cannot guarantee positive expected gain in every market.

Gold: can history reveal the market's rule?

Learning in the limit

After each new observation, a learner proposes a program explaining the whole sequence. Success means eventually settling on one correct program and never changing it again. The learner need not know when it has succeeded.

Gold's negative result

No algorithm achieves this for every computable sequence of zeros and ones. For each learner, some computable history defeats this requirement. Failure means no eventual correct identification, rather than a specified number of losing trades.

Restrictions can make learning possible

If an algorithm can list the allowed rules, and each rule computes every requested term in finite time, identification in the limit is possible. What matters is which market rules the learner admits.

Sources: E. M. Gold, *Limiting recursion* (1965); *Language identification in the limit* (1967). Related prediction construction: Legg, *Is there an Elegant Universal Theory of Prediction?* (2006).

Turing: will the price ever rise?

An artificial market controlled by a program

Take any program with a fixed input. Start the price at 1. Execute one instruction of the program per day.

- ▶ While the program keeps running, the price stays at 1.
- ▶ If the program stops, the price becomes 2 and stays there.

The prices simply indicate whether the program has stopped.

We can calculate the price on any specified day

Program's behaviour	Day 1	Day 2	Day 3	Day 4	...
Stops at instruction 3	1	1	2	2	...
Never stops	1	1	1	1	...

For day 100, simulate at most 100 instructions. If the program has stopped, report 2; otherwise, report 1. We never need to wait indefinitely.

But will the price ever rise?

This asks whether the program will *ever* stop: the *Halting Problem*. Turing proved that no algorithm can always answer this correctly for every program and input.

Therefore, even with the price generator's code, no algorithm can decide for every such market whether its price will ever rise.

Radó's Busy Beaver: how long must we wait?

A finite maximum with no computable general bound

Fix a programming language. Among programs of at most n bits that eventually stop, take the longest running time. This is the runtime *Busy Beaver* value, written $BB(n)$. Here n measures program length, not elapsed time.

Return to the price 1 that may become 2

A short program can run for an extraordinarily long time before stopping. Its market can therefore remain flat through a long observation period and change only later.

Why no waiting rule suffices

No computable deadline based only on program length covers every eventual halt. Such a deadline would solve Turing's problem: wait until it expires. Some finite cases can be settled, but they give no general extrapolation rule.

Sources: T. Radó, *On non-computable functions* (1962); G. J. Chaitin, *Computing the busy beaver function* (1987).

Martin-Löf: randomness still permits finite wins

Random relative to which model?

Use a binary record with independent, equally probable zeros and ones. A *Martin-Löf-random* infinite record escapes every algorithmically specified statistical test whose exceptional sets have probabilities shrinking effectively to zero.

The betting consequence

In the corresponding fair betting game, no computable strategy that keeps its capital nonnegative can make that capital unbounded on such a record. *Fair* means that, conditional on the history, the average next capital equals the current capital.

What remains possible

Finite winning streaks, apparent trends, and correct forecasts still occur. Every finite record has both computable and algorithmically random continuations. A finite sample alone cannot certify randomness.

Sources: P. Martin-Löf, *The definition of random sequences* (1966); J. Ville, *Étude critique de la notion de collectif* (1939).

Ramsey: order appears even without a predictive law

An unavoidable pattern

Join every pair among six objects and color each link red or blue.
Whatever the coloring, three objects have all three links the same color.
This is a basic finite case of Ramsey's theorem.

Repeated symbols at equally spaced dates

Van der Waerden's theorem gives a related result: in a sufficiently long record with finitely many symbols, some symbol repeats at equally spaced positions for any prescribed finite number of repetitions.

The finite-pattern trap

The selected dates need not be consecutive, the returns need not be positive, and the pattern need not continue. Calude and Longo emphasize that large data sets force some regularities even when the records resist compression. Such patterns coexist with Martin-Löf randomness.

Sources: Graham, Rothschild & Spencer, *Ramsey Theory* (1990); [Graham & Spencer \(1990\)](#); [Calude & Longo, *The deluge of spurious correlations in big data* \(2017\)](#).

Time reversal as a search for counterexamples

Hold the trading rule fixed, change the record

Test it on the illustrative prices 100, 110, 120 and then rerun it from scratch on 120, 110, 100. At every date it may use only the history available on that run. Reversing the original orders would give it information from the future.

One failure refutes an all-path claim

If both records belong to the claimed market class, a nonpositive gain on either refutes strict profit on every path. Reversal alone does not prove that a rule must fail on one of the pair. It supplies a stress test, whereas diagonalization supplies the counterexample.

Sources: KS, "A Practical Robustness Diagnostic: Time Reversal", Reversal Falsification Test.

Beyond time reversal: reshuffling the record

Algorithmic and random permutations

A *permutation* rearranges the same finite record of discounted price levels. Use a fixed algorithm or a random shuffle. Rerun the trader from scratch, giving it only the past available in the new order.

A necessary condition for a universal profit guarantee

A strategy claiming strict profit on every permitted path **must remain profitable under every reshuffling that stays within its claimed market class**. The amount of profit may change. One zero or negative gain refutes that universal claim.

What this requirement means

A strategy exploiting a particular trend may fail when reshuffling destroys that trend. Such failure limits its scope. Conversely, passing a finite collection of shuffles cannot establish universal profitability.

Literary analogy: Greg Egan's *Permutation City* (1994) explores the ordering of computational states.

Sources: Greg Egan, *Permutation City* (1994); Egan's *Dust Theory FAQ*, questions 1–2.

Conclusions

Computable diagonal countermarket

For every total deterministic computable trading rule with rational positions, there exists a positive computable price path on which its self-financing gain is nonpositive at every finite horizon.

Scope of the result

The countermarket depends on the chosen rule. Every nonzero position incurs a loss, while zero holdings yield zero gain. Thus no such rule guarantees strictly positive discounted gain on every positive computable price path.

Finite profitability

Information, luck, or favorable market conditions can produce finite profits. These remain compatible with adverse continuations and do not establish a universal guarantee.

Thank you for your attention